Which Wrong Pages Should an Event Linker Train On?

Choosing the wrong examples for Event Linking

Rodrigo Quezada
COMPSCI 2242, Fall 2026

Draft, November 13, 2026

Abstract

A news story says that a player won the Rising Star award in 1996. A reader who wants to know more needs the Wikipedia page for that award in that year, not the page for 1998. Event linking is the task of finding that page for an event mentioned in a text. A system learns the task from examples. For each example it sees the right page and a few wrong pages, and it learns to prefer the right one. Which wrong pages should it see? Pages that are obviously wrong teach it little. Pages that are almost right can mislead it.

I test four rules for choosing the wrong pages in EVELINK, the event linking system of Yu et al., and I give each rule the same amount of training. One rule uses the right pages of other examples, as EVELINK does. One takes random pages from the 1,000 that the system already ranks highest. One takes only the seven highest. The last is ReSa2, a recent rule that picks pages that score close to the right page and resemble it.

First, ReSa2 works best. The right page is among the first ten results for 93.3% of held-out mentions, compared with 92.2% for random pages from the 1,000 and 69.6% for EVELINK's rule. Second, I measure which pages each rule picks and how much its picks change from run to run. Third, I prove how little an obviously wrong page can teach, and I check the proof in Lean 4. Fourth, the seven highest pages are the worst of the three new rules, and with faster training they break the system. No rule reduces wrong-year mistakes, and none improves the results on news text.

1 Introduction

EVELINK is a system that reads a mention of an event and tries to find the Wikipedia page for that event. Yu and colleagues introduced it in 2023 [Yu et al.(2023)]. You give it a phrase like "won the Rising Star award," along with the sentence around it. EVELINK uses that context to work out which award the writer is talking about.

Take this sentence. "He won the Rising Star award in 1996, in his first season with Adelaide."

The page we want is "1996 AFL Rising Star." A page about the 2004 Indian Ocean earthquake is easy to rule out. The page for the 1998 AFL Rising Star is harder. It is the same award. The year is what tells us which page to link to. The Event Linking paper calls this mistake a time error. Its other common mistake is a hierarchy error, where the system picks a larger event that contains the right one, or a smaller event inside it. In both cases the wrong page shares most of its words with the correct page.

EVELINK makes that decision in two stages (section 2). The first stage searches all of Wikipedia and pulls out 100 pages that might fit. It is called the *retriever*. The second stage reads the mention

next to each of those 100 pages and puts them in a new order. It is called the *reranker*. If the retriever leaves out the 1996 page, the reranker cannot bring it back.

That is where this project starts. During training, the retriever sees the correct page next to a few wrong pages. I wanted to know which wrong pages help it most. A rule that picks the wrong pages is called a *sampler*, because it usually picks them at random from some set. The sampler I study is ReSa2 [Li et al.(2025)]. I ask two questions.

1. Which pages does ReSa2 pick? For each page I want the probability that it is picked, the number of different pages picked over many tries, and how much the picks change between tries.
2. With the same amount of training, does ReSa2 put the correct page higher in the results than EVELINK's own sampler does?

The paper has four results. First, I train the EVELINK retriever with four samplers, three times each, and compare them. Second, I measure what each sampler picks. Third, I prove a bound on the training loss and three counting facts about the samplers, and I check the proofs in Lean 4. Fourth, I describe a failure: with the hardest wrong pages and faster training, the retriever stops working and does not recover. The code and the logs are in the project repository.

2 Event linking: task, system, and evaluation

Task. Event linking connects an event mention in text to the Wikipedia page about that event. A mention is a word or short phrase, together with the text around it. For example, in "He won the Rising Star award in 1996," the mention should lead to the page for the 1996 AFL Rising Star. The year matters. A page for the same award in 1998 is still a wrong answer.

Formally, a mention $m = (d, [a, b])$ is a document d and the positions a to b of the words in it that refer to the event. Let $\mathcal{E}$ be the set of Wikipedia pages that the system can link to. Event linking is the function

$$\text{link} : m \longmapsto e_m^* \in \mathcal{E} \cup \{\text{NIL}\}, \tag{1}$$

where e_m^* is the page about the event, and NIL means that no page describes it.

Data. Yu et al. built the training data from links that Wikipedia editors had already made. The data contains 66,426 training mentions, 16,692 validation mentions, and 19,266 test mentions. The system can link to $|\mathcal{E}| = 5{,}870{,}755$ Wikipedia pages. There is also a New York Times dataset with 1,762 mentions. Of those, 769 have a corresponding page. Each mention is a verb ("won") or a noun ("the award"). Verbs are split by whether the same verb and the same answer appear in the training data. Nouns are split into easy and hard by how much the mention looks like the title of its page.

System. Reading each mention together with almost six million pages would cost too much. EVELINK uses two stages instead. The first stage, the *retriever*, encodes the mention and the pages separately. It compares their vectors and returns the 100 pages with the highest scores. The second stage, the *reranker*, reads the mention with each candidate and selects an answer:

$$\text{retrieve:} \quad R_K(m) = \text{top-}K_{e \in \mathcal{E}}\ s(m, e), \tag{2}$$

$$\text{rerank:} \quad \hat{e}_m = \operatorname*{arg\,max}_{e \in R_K(m)}\ r(m, e). \tag{3}$$

Here $s(m, e)$ is the retriever's score (section 4), $r(m, e)$ is the reranker's score, and $K = 100$. This makes the first stage a limit on the full system. If the correct page is missing from its list, the reranker cannot select it.

Evaluation. The full system is measured by *accuracy*: how often its final answer is correct. With V the set of mentions used for scoring, and $\mathbf{1}[\cdot]$ equal to 1 when the statement inside is true and 0 otherwise,

$$\text{Acc} = \frac{1}{|V|} \sum_{m \in V} \mathbf{1}[\hat{e}_m = e_m^*] \,. \tag{4}$$

The released system gets 82.87% on the Wikipedia test set and 55.33% on the New York Times set. My reproduction gets 82.88% and 55.33%.

This project changes the first stage, so I measure *recall at 10*. This is the share of mentions whose correct page appears in the first 10 results. In general,

$$\text{Recall@}K = \frac{1}{|V|} \sum_{m \in V} \mathbf{1}[e_m^* \in R_K(m)] \,. \tag{5}$$

It measures whether the retriever supplies the right candidate, even if that candidate is not first. The released retriever gets 77.7% on the Wikipedia validation set. I also report recall at 1 and at 100.

Where the mistakes are. Yu et al. describe two important mistakes. A *time mistake* selects the same event in another year. A *hierarchy mistake* selects a larger event that contains the correct event, or a smaller event inside it. Selecting a war when the mention refers to one battle is a hierarchy mistake. These pages are hard negatives because they share much of the right page's meaning and wording. The retriever must learn which details identify the event.

3 Related work

This project builds on EVELINK, the event linking system released by Yu et al. [Yu et al.(2023)]. EVELINK builds on BLINK [Wu et al.(2020)], a two-stage system for linking entities to Wikipedia. I keep the EVELINK task, data, and two-stage structure. The experiment changes how wrong pages are selected for retriever training.

The cheapest negatives are answers already present in the training batch. DPR [Karpukhin et al.(2020)] uses this approach. It avoids a separate search for wrong pages, but those pages may be easy to distinguish from the correct answer.

Another approach takes the pages that the model scores highest. ANCE [Xiong et al.(2021)] uses high-scoring negatives and refreshes them during training. These pages provide harder comparisons. The risk is that a high-scoring page may be a valid answer that was not labeled. Training then pushes down a page that should not be treated as wrong. RocketQA [Qu et al.(2021)] addresses this problem with a second model that filters potential false negatives.

SimANS [Zhou et al.(2022)] uses random sampling but prefers pages whose scores are close to the correct answer's score. ReSa2 [Li et al.(2025)] adds another selection step. After the first sample, it keeps pages that are most similar to the correct answer. In the released ReSa2 code, the process

starts with 1,000 candidates. The first step selects 500. The second keeps 75. Training then uses 7 pages sampled from those 75.

My contribution is to test this method on event linking and examine what its two steps select. I compare it with in-batch negatives, uniform sampling, and a top-score rule. The aim is to measure both the retrieval result and the selection process that produces it.

4 The retriever and its training loss

The retriever of equation 2 uses two copies of BERT, a neural network that turns a text into a list of numbers. Such a network is called an *encoder*. One encoder reads the mention and its context. The other reads the Wikipedia page. Each one turns its input into a vector with 1,024 numbers. I call the mention vector u_m and the page vector v_e.

To score a page, I take the dot product. I multiply the first number in one vector by the first number in the other, do the same for the rest, and add up the products:

$$s(m, e) = u_m \cdot v_e = \sum_{j=1}^{1024} u_{m,j}\, v_{e,j}. \tag{6}$$

The retriever puts pages with higher scores first. The two encoders work separately, so I can compute the page vectors ahead of time. When a new mention comes in, I encode it once and compare that vector with the stored page vectors.

During training, I give the retriever the correct page and seven wrong pages. I call the wrong pages *negatives*. If the correct page scores s^+ and the negatives score s_1 through s_7, the *loss*, the number that training tries to make small, is

$$L = -\log \frac{e^{s^+}}{e^{s^+} + \sum_{i=1}^{7} e^{s_i}}. \tag{7}$$

The fraction is the correct page's share of the total after I exponentiate the eight scores. When the correct page gets almost all of that share, the loss is close to zero. When a wrong page scores about as high as the correct one, it takes a bigger share and the loss goes up.

To see the difference, suppose the correct page scores 10 and there is just one negative. If that negative scores -10, the loss is about two billionths. If it scores 9.5, the loss is about 0.47. The correct page has the same score in both cases. All I changed was the wrong page.

The math puts a bound on this. Write $g_i = s^+ - s_i$ for the gap by which negative i scores below the correct page.

Lemma 1 (Easy negatives add little)**.** *For any $n \geq 1$ and real scores $s^+, s_1, \ldots, s_n$, the loss of equation 7 with n negatives satisfies*

$$L = \log\Bigl(1 + \sum_{i=1}^{n} e^{-g_i}\Bigr) \leq \sum_{i=1}^{n} e^{-g_i}, \qquad \text{and} \qquad L \leq n\, e^{-g} \ \text{ if every } g_i \geq g. \tag{8}$$

Proof. Divide the numerator and the denominator of the fraction in equation 7 by e^{s^+}: $L = -\log\bigl(1/(1+\sum_i e^{s_i - s^+})\bigr) = \log(1+\sum_i e^{-g_i})$. For $x \geq 0$, $\log(1+x) \leq x$, so the first inequality holds. If every $g_i \geq g$ then each $e^{-g_i} \leq e^{-g}$, and the second follows. □

So a negative that scores g points below the correct page adds at most e^{-g} to the loss. At $g = 20$ that is about 2×10^{-9}. At $g = 0.5$ it is about 0.61. I checked Lemma 1 in Lean 4 (section 5.1).

I can also ask how much the loss changes if I raise one negative's score.

Lemma 2 (The push on a negative)**.** *The derivative of L with respect to the score of negative i is that negative's share of the total,*

$$p_i = \frac{\partial L}{\partial s_i} = \frac{e^{s_i}}{e^{s^+} + \sum_{j=1}^{n} e^{s_j}} \in (0, 1), \tag{9}$$

and $\partial L/\partial s^+ = -\sum_i p_i$.

Proof. Write $Z = e^{s^+} + \sum_j e^{s_j}$, so $L = \log Z - s^+$. Then $\partial L/\partial s_i = e^{s_i}/Z$ and $\partial L/\partial s^+ = e^{s^+}/Z - 1 = -\sum_i e^{s_i}/Z$. □

So if the retriever already puts the earthquake page far below the 1996 award, that page contributes almost nothing to the update. If the 1998 award scores close to the 1996 award, the loss puts more pressure on the model to separate them.

The update itself moves the mention vector.

Corollary 1 (The update on the mention vector)**.** *With the page vectors fixed, the gradient of L with respect to the mention vector is*

$$\nabla_{u_m} L = \sum_{i=1}^{n} p_i \, (v_{e_i} - v_{e^+}), \tag{10}$$

Proof. By the chain rule through equation 6, $\nabla_{u_m} L = \sum_i (\partial L/\partial s_i)\, v_{e_i} + (\partial L/\partial s^+)\, v_{e^+} = \sum_i p_i v_{e_i} - \big(\sum_i p_i\big) v_{e^+}$ by Lemma 2. □

Equation 10 is a weighted sum of the differences between each negative's page vector and the correct page vector. A negative that scores far below the correct page has $p_i \approx 0$ and moves nothing. A negative that scores close to it has a large p_i, and the step is along $v_{e_i} - v_{e^+}$. If the two pages are near-duplicates, that difference is small. Section 6.5 returns to this.

In the experiment, I keep the loss and the number of negatives fixed. I change which seven pages the model sees.

5 How I chose the seven pages

I compared four samplers. The first uses the correct pages of the other seven mentions that are trained at the same time. This is EVELINK's own sampler, and it is called *in-batch*. For the other three, I started with the pages that the released retriever ranked highest for that mention. I built a pool of up to 1,000 candidates and excluded the correct page from sampling. I call these 1,000 pages the *pool*. Every sampler gets the same three things. It gets the score s_i of each pool page, the score s^+ of the correct page, and the similarity $\sigma_i = v_{e_i} \cdot v_{e^+}$ between each pool page and the correct page.

Uniform sampling gives every page in the pool the same chance, $k/N = 7/1{,}000$. *Top-score* sampling always takes the seven pages with the highest scores. Those seven are picked with probability 1 and the rest with probability 0. ReSa2 makes the choice in two stages.

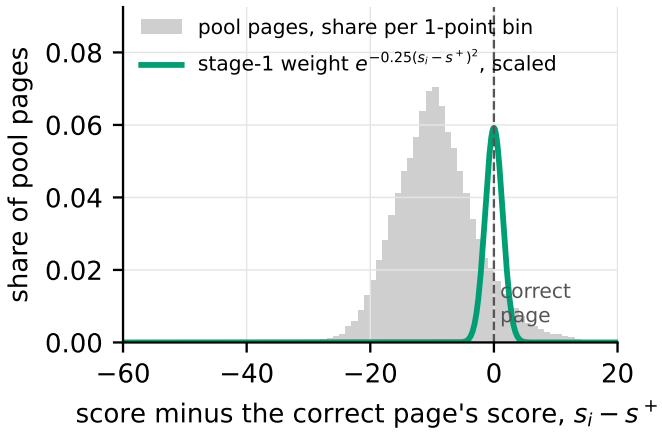


Figure 1: The weight of equation 11 (green, scaled to fit) and how far the pool pages score from the correct page (gray, 2,000 validation mentions). Most pool pages score 5 to 20 points below the correct page. The weight is close to zero for pages more than 4 points away.

First, ReSa2 draws 500 pages without replacement. It gives the most weight to pages that score close to the correct page:

$$w_i = \exp\!\left(-\tau\,(s_i - s^+)^2\right), \qquad \tau = 0.25. \tag{11}$$

It draws one page with probability $w_i / \sum_j w_j$, removes it, and draws again from the pages that are left. The weight drops as the score moves away from s^+ in either direction. Figure 1 shows the weight next to the real scores. The scores are about 550, and most pool pages are 5 to 20 points below the correct page. The weight is close to zero for any page more than 4 points away. So stage 1 first takes the few pages near s^+, and then fills up to 500 with nearly random pages from the rest. Then ReSa2 compares the 500 pages that are left with the correct page itself. It keeps the 75 pages whose vectors are most similar to the correct page vector and draws seven of those uniformly.

That last draw gives a simple bound. Once a page makes the final 75, it has a 7-in-75 chance of being used. Suppose a page is labeled as wrong but is in fact a valid answer. Then I can bound how often ReSa2 trains against it. With equation 9, I can also bound its expected push on the model:

$$\begin{aligned} \mathbb{E}[p_i] &\le P(\text{ReSa2 picks } i) \cdot \frac{e^{s_i}}{e^{s^+} + e^{s_i}} \\ &\le \frac{7}{75} \cdot \frac{e^{s_i}}{e^{s^+} + e^{s_i}}. \end{aligned} \tag{12}$$

5.1 What the Lean proofs establish

Lean is a proof assistant. It checks whether a formal proof follows from its definitions and assumptions. I use it to check four claims about the loss and the selection rules. These claims help explain the experiment, but they do not prove which method will perform best. The files are in Appendix B.

The first claim is Lemma 1. It bounds the loss when all negatives score far below the correct page. With seven negatives, each at least 10 points below the positive, the loss is at most $7e^{-10}$, or about 0.0003. The model already separates these pages well. They contribute very little loss.

The second claim gives a counting limit. Suppose a candidate pool has A pages that score above the correct page. I call such a page a *possible false negative*.

Proposition 1 (A bound for every sampler)**.** *Let the pool contain A possible false negatives. Any selection of k distinct pages from the pool contains at most* $\min(k, A)$ *of them.*

Proof. The selection has k pages, and it is a sub-list of the pool, so it cannot contain more pages above s^+ than the pool does. □

If there are three such pages and the rule selects seven, it can select at most three. The third claim says that the top-score rule reaches this limit.

Proposition 2 (Top-score reaches the bound)**.** *The top-score rule selects exactly* $\min(k, A)$ *possible false negatives.*

Proof. Sorting by score puts every page above s^+ before every page below it, so the first k pages are the $\min(k, A)$ pages above s^+ followed, if $k > A$, by $k - A$ pages below it. □

In the same example, it selects all three, every time. Together, the second and third claims show that no rule selects more above-positive pages than the top-score rule.

An above-positive page is not necessarily a false negative. It may simply be a wrong page that the model scores too highly. But if some of these pages are valid answers without labels, the top-score rule selects them as often as possible under this count.

The fourth claim concerns ReSa2's final draw. It selects seven of the remaining 75 pages uniformly.

Proposition 3 (The 7/75 share)**.** *Of all 7-page subsets of a set of 75 pages, the fraction that contains one fixed page is* $7/75$*; equivalently,* $75\binom{74}{6} = 7\binom{75}{7}$*. Hence*

$$P(\textit{ReSa2 selects } i) = \tfrac{7}{75}\, P(i \textit{ is among the 75 kept by stage 2}),$$

and equation 12 follows.

Proof. A subset that contains the fixed page is the fixed page plus 6 of the other 74 pages, so there are $\binom{74}{6}$ of them out of $\binom{75}{7}$, and $\binom{74}{6}/\binom{75}{7} = 7/75$. The identity is checked by computation in Lean. □

Each page has a 7/75 chance of being used, about 9.3%. This applies only after the page reaches the final pool. It does not give the page's chance of surviving the first two steps.

The loss bound uses Mathlib, the Lean library of mathematics. The other three claims use core Lean. The derivative of the loss and the gradient result in Lemma 2 and Corollary 1 are proved by hand. They are not checked in Lean. The first sampling step also has a separate numerical check: exact selection probabilities for pools of up to eight pages agree with a simulation of the code to within 0.015.

Table 1: Recall on the 16,692 validation mentions after 1,500 updates. Each cell is the mean of three runs ± their standard deviation. "Released" is the model before my training. The three runs are listed one by one in Appendix A.

Sampler	Top-1	R@10	R@100
Released	42.6	77.7	93.1
In-batch	33.6 ± 1.9	69.6 ± 1.8	89.8 ± 1.3
Uniform	64.3 ± 0.2	92.2 ± 0.1	98.4 ± 0.1
Top-score	65.3 ± 1.2	87.2 ± 0.9	95.9 ± 0.6
ReSa2	**69.5** ± 0.3	**93.3** ± 0.1	**98.5** ± 0.1

6 Experiments

6.1 Setup

I started each run from the released EVELINK model. I kept the page encoder fixed and trained only the mention encoder. Since the page vectors never change, the pool of each mention and its scores can be computed once, before training.

Each run made 1,500 updates. One update uses 32 mentions, each with seven negatives. The learning rate, which sets the size of each update, was 10^{-5}. It rises from zero over the first 10% of the updates. The other settings are those of EVELINK. Training has some randomness, so I ran each of the four samplers three times with different random seeds. That gives twelve runs. Each run took about 56 minutes on one NVIDIA A10G GPU.

Then I ranked all 5.87 million pages for each of the 16,692 validation mentions and computed recall at 1, 10, and 100 (equation 5). The measure I focus on is recall at ten. If the correct page appears anywhere in the first ten results, I count that mention as a hit.

Before the experiments I checked that I could repeat the published results. The released retriever and reranker gave me an accuracy of 82.88% on the Wikipedia test set, and the paper reports 82.87%. On the New York Times set I got 55.33%, the same as the paper. I did not use the test set for anything else.

6.2 What changed in the rankings

Table 1 gives the results. The released retriever got 77.7 percent recall at ten. After fine-tuning, in-batch sampling got 69.6. Uniform sampling from the pool got 92.2. Taking the seven highest-scoring negatives got 87.2. ReSa2 got 93.3.

Most of the improvement came from drawing negatives from the pool. ReSa2 added 1.1 percentage points over uniform sampling at ten. The difference was larger at rank one. With uniform sampling, the correct page came first 64.3 percent of the time. With ReSa2, it came first 69.5 percent of the time. For both samplers the three runs differ by 0.1 to 0.3 points, so the gap between them is much larger than the differences between runs.

In-batch sampling finished below the released model. I think the reason is the small number of negatives. Seven random pages are almost always easy, and by Lemma 1 easy pages add almost nothing to the loss. The original training of EVELINK used many more pages at each update and ran for longer.

Table 2: What each sampler picks, over 500 validation mentions and 200 draws per mention. The first column splits the seven negatives of one draw by how similar they are to the correct page: among the 25 most similar pool pages, the next 50, the next 225, or the rest. The overlap of two draws is the number of pages they share divided by the number of pages in either.

Sampler	Negatives by similarity rank top 25 / 25–75 / 75–300 / rest	Distinct pages (of 1,000)	Overlap of two draws	Score gap to correct page
Uniform	0.17 / 0.34 / 1.55 / 4.95	756	0.00	−13.3
Top-score	4.13 / 1.07 / 1.17 / 0.62	7	1.00	−1.4
ReSa2	2.15 / 3.98 / 0.87 / 0.00	84	0.05	−8.6

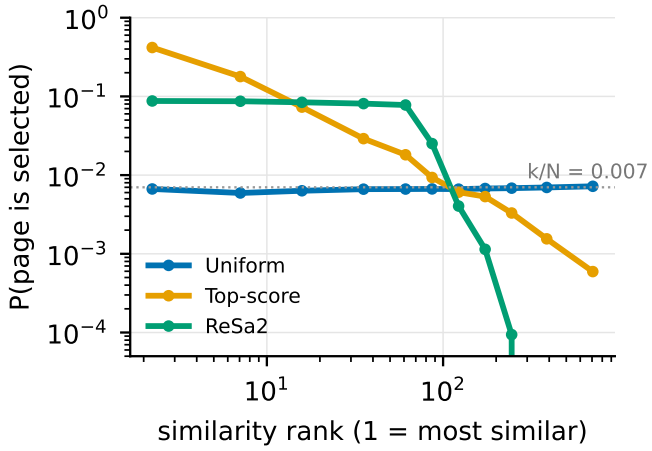


Figure 2: The probability that a pool page is picked, by how similar it is to the correct page. Rank 1 is the most similar page. Both axes are logarithmic. Uniform gives every page the same chance. ReSa2 gives about 0.09 to each of the 75 most similar pages and picks nothing beyond rank 300.

6.3 Which pages the samplers pick

I also wanted to see which pages the samplers actually picked. I drew negatives 200 times each for 500 validation mentions. Table 2 and Figure 2 give the result.

ReSa2 picked about six of its seven negatives from the 75 pages most similar to the correct page. For uniform sampling, the average was about half a page. Top-score gave me the same seven pages every time. Two ReSa2 draws shared very few pages, even though both tended to draw from the same part of the pool. Over 200 draws, uniform sampling used 756 different pool pages. ReSa2 used 84, and top-score used 7.

The last column says how hard the negatives are. On average, ReSa2's negatives score 8.6 points below the correct page. Uniform negatives score 13.3 points below it, so they are easier. Top-score negatives score only 1.4 points below it, so they are the hardest. Figure 2 shows the same picture page by page. Top-score puts most of its picks on the first few pages, which are the near-copies of the correct page.

6.4 Mention types, error types, and news text

The proposal included three checks: performance by mention type, time and hierarchy mistakes, and transfer to news text. ReSa2 improves the first check. It does not reduce time mistakes or

Table 3: Recall at ten on the validation set by kind of mention (mean of three runs ± standard deviation). From left to right the columns hold 1,814, 2,585, 3,947, 4,173, and 4,173 mentions.

	Verb			Noun	
Sampler	Seen	Unseen form	Unseen	Hard	Easy
Released	76.1	77.3	82.9	68.9	82.6
In-batch	65.2 ± 1.5	69.0 ± 1.3	75.0 ± 0.6	58.0 ± 1.5	78.4 ± 3.9
Uniform	98.8 ± 0.2	91.3 ± 0.2	88.3 ± 0.1	88.8 ± 0.2	97.1 ± 0.3
Top-score	96.5 ± 0.2	83.2 ± 1.5	79.6 ± 1.2	84.9 ± 1.0	95.2 ± 0.5
ReSa2	**99.1** ± 0.0	**92.3** ± 0.1	**90.2** ± 0.1	**90.3** ± 0.2	**97.5** ± 0.2

Table 4: Mentions whose first result is wrong, as a percentage of all 16,692 validation mentions, split by kind of mistake.

Sampler	Wrong first result	Time	Hierarchy	Other
Released	57.4	1.8	11.6	44.0
In-batch	66.4 ± 1.9	1.9 ± 0.1	11.7 ± 0.6	52.8 ± 1.4
Uniform	35.7 ± 0.2	2.1 ± 0.1	5.5 ± 0.2	28.1 ± 0.2
Top-score	34.7 ± 1.2	2.1 ± 0.1	3.0 ± 0.1	29.6 ± 1.2
ReSa2	30.5 ± 0.3	2.0 ± 0.1	4.4 ± 0.2	24.0 ± 0.1

improve retrieval on news text relative to the released model.

Table 3 gives recall at 10 by mention type. Hard noun mentions are the weakest category for the released retriever, with 68.9%. ReSa2 raises this to 90.3%. Uniform sampling reaches 88.8%, while top-score reaches 84.9%.

ReSa2 leads uniform sampling in every mention category. Its largest lead is on unseen verbs: 90.2% against 88.3%, a difference of 1.9 percentage points. This is consistent with a benefit from more selective negatives when the mention is less familiar. The table alone does not establish that explanation.

The error breakdown in Table 4 is less favorable to the original motivation. Time mistakes remain close to 2% of all validation mentions. The released model has 1.8%, uniform and top-score have 2.1%, and ReSa2 has 2.0%. There is no reduction here.

One possible reason concerns where wrong-year pages score. A wrong-year page can score above the correct page. ReSa2's first step prefers scores near the positive score, so it can reduce selection of pages far above it. If those pages need correction, selecting them less often can leave the mistake unresolved. This is a possible mechanism, not proof that it explains the observed result.

Top-score produces the fewest hierarchy mistakes: 3.0% of validation mentions, compared with 4.4% for ReSa2. ReSa2 still has fewer errors overall. Its first result is wrong for 30.5% of mentions, compared with 34.7% for top-score. Its advantage comes from the broader error count, especially the "other" category.

The news results in Table 5 show a transfer limit. On the 769 New York Times mentions with a page, the released retriever gets 33.6% recall at 10. ReSa2 gets 30.9%. Recall at 100 also falls, from 57.3% to 55.9%. ReSa2 performs better than the other retrained rules on this dataset, but it does not beat the released model. The gains after 1,500 Wikipedia training updates do not carry over to news text.

The error labels are approximate. A title rule identifies the same title with a different year as

Table 5: Recall on the 769 New York Times mentions that have a Wikipedia page (mean of three runs ± standard deviation).

Sampler	Top-1	Recall@10	Recall@100
Released	**13.0**	**33.6**	**57.3**
In-batch	8.5 ± 1.1	24.4 ± 1.4	44.8 ± 1.7
Uniform	9.8 ± 0.7	27.4 ± 1.0	50.8 ± 1.6
Top-score	4.6 ± 0.7	14.4 ± 1.4	26.8 ± 1.8
ReSa2	11.5 ± 0.1	30.9 ± 1.5	55.9 ± 0.4

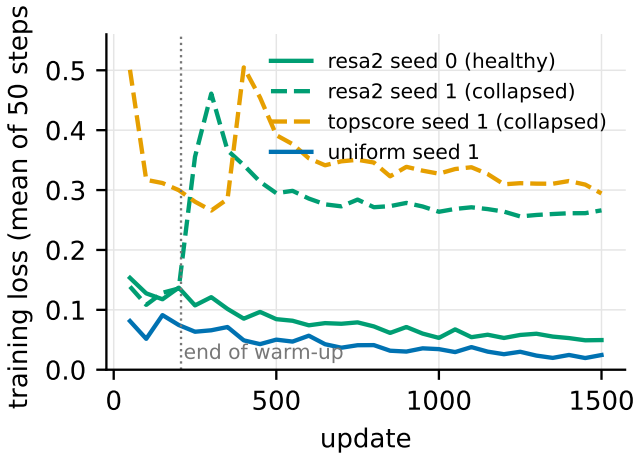


Figure 3: Training loss at the higher learning rate for two healthy runs (solid) and the two collapsed runs (dashed). The dotted line marks the update at which the learning rate reaches its highest value. The collapsed runs jump once and stay at about 0.3.

a time mistake. It identifies one title's words contained in another title as a hierarchy mistake. Everything else is "other." No person reviewed every case. These labels measure title patterns, which can miss real relationships or assign a relationship that the titles alone do not establish.

6.5 What happened at the higher learning rate

My first twelve runs used a learning rate three times higher, 3×10^{-5}. One ReSa2 run and one top-score run collapsed. Figure 3 shows the training loss. It jumped once, just after the learning rate reached its highest value, from about 0.13 to about 0.3, and it stayed there. When I looked at the scores, I found that the mention encoder gave almost every page a score of about two. It gave roughly the same score to the correct page and to a random page. A healthy model gives about 550 to the correct page and 545 to a random one. The weights stayed finite. The loss over the eight training pages could still look modest while the ranking across millions of pages had fallen apart.

One possible explanation is in Corollary 1. When a negative scores close to the correct page, it gets a substantial share of the training signal. If the two page vectors are also similar, the update depends on the small difference between them. My hypothesis is that repeated updates at the higher learning rate weakened the part of the mention vector that lined up with the page vectors. The scores are consistent with that explanation. Uniform and in-batch runs, whose negatives are far from the correct page, never jumped.

I tried training the collapsed model for another 1,500 updates. I tried ReSa2 negatives in one run

and uniform negatives in another. Recall at ten stayed at zero in both. Then I started again from the released model with a learning rate of 10^{-5}. All twelve runs trained without collapsing, and every sampler scored better than at the higher rate (Appendix A). Those twelve runs are the ones reported above.

7 Conclusion

I started with the difference between the 1996 award and the 1998 award. The results make that example more specific for me. I want the retriever to see pages that are close enough to make it pay attention to the year. I also want it to see a different set of pages the next time it trains on that mention. In these runs, ReSa2 gave me that variety and put the correct page first more often than any of the other samplers I tested. It did not fix everything. The wrong-year errors that started this paper are as frequent after training as before, with every sampler, and on news text the released model still retrieves better than any of the fine-tuned ones.

Limitations

The new comparisons use validation data. I did not use the test set to evaluate the trained variants. The results therefore remain validation results.

The page encoder is frozen, and each mention's candidate pool comes from the released model's first 1,000 results. The page vectors and candidate membership stay fixed. Mention-side training can still change their scores. Training both encoders or refreshing the pools could produce different results.

Each rule receives 1,500 updates with seven negatives per example. The study uses three runs for each rule and two learning rates. This supports a comparison under one training budget. It does not establish the best rule across tasks, budgets, or model settings.

Two runs failed at the higher learning rate. Their failure does not, by itself, establish the cause. Any explanation of those failures must remain separate from the observed result.

The fixed candidate pools also limit what training can address. For about 2% of mentions, the correct page is absent from the first 1,000 candidates. The experiments do not explore negatives outside those pools.

The Lean proofs cover specific mathematical claims. They do not verify the training code, the data, the experimental results, or the assumption that an above-positive page is a false negative. The derivative and gradient arguments remain outside Lean.

Finally, the error analysis uses title rules rather than a full human review. Its categories are useful for an initial comparison, but they are not a complete account of event relationships.

How I used AI

AI assistance was a large part of this project. Before the proposal, I used Codex to search for papers from researchers in the instructor's academic circle about linking documents, which led to Event Linking, and then for a more recent negative sampler, which led to ReSa2 and SimANS. It also wrote "learning versions" of the papers that explained them through the concepts of this

course (conditional probability, indicator variables, expectation) with small examples; I read those first and the full papers second.

During the project I used Claude, through an agent that ran on my computer and on a cloud GPU. The agent did substantial implementation work. It set up the cloud environment, changed the implementation to fit a 24 GB GPU, implemented the sampling rules, ran the 24 training runs, and produced tables, figures, and Lean proofs.

That work had failures. These included out-of-memory errors, a full disk, a run that reached its time limit, and a bug in the original validation code. The process required checks and corrections.

The checks included reproducing the released system's accuracy, tests for the sampling rules, three runs for each rule, and Lean checks for the formal claims. These checks cover different parts of the work. Reproducing the baseline does not verify every later experiment, and Lean does not verify the training implementation.

AI also helped write the paper. I narrated the introduction, sections 4 and 5, the main results, and the conclusion, and the agent turned the narration into plain technical English. Sections 2, 5.1 and 6.4, the related work, the limitations, and this section were generated from a sheet of questions, facts, and results. I asked an agent to turn that material into prose in my voice. They were not transcribed from my spoken narration.

The repository contains an AI-usage log with an entry for each session. The prompts and outputs are also retained.

References

[Yu et al.(2023)] Xiaodong Yu, Wenpeng Yin, Nitish Gupta, and Dan Roth. 2023. Event Linking: Grounding Event Mentions to Wikipedia. In *Proceedings of EACL*.

[Li et al.(2025)] Muyang Li, Zihan Wang, Sijia Chen, Yijun Chen, Jiayu Li, Yiming Qiao, Xinyi Li, and Bo Ji. 2025. ReSa2: A Two-Stage Retrieval-Sampling Algorithm for Negative Sampling in Dense Retrieval. In *Proceedings of ACML*, PMLR 304.

[Wu et al.(2020)] Ledell Wu, Fabio Petroni, Martin Josifoski, Sebastian Riedel, and Luke Zettlemoyer. 2020. Scalable Zero-shot Entity Linking with Dense Entity Retrieval. In *Proceedings of EMNLP*.

[Karpukhin et al.(2020)] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of EMNLP*.

[Xiong et al.(2021)] Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul Bennett, Junaid Ahmed, and Arnold Overwijk. 2021. Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval. In *Proceedings of ICLR*.

[Qu et al.(2021)] Yingqi Qu, Yuchen Ding, Jing Liu, Kai Liu, Ruiyang Ren, Wayne Xin Zhao, Daxiang Dong, Hua Wu, and Haifeng Wang. 2021. RocketQA: An Optimized Training Approach to Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of NAACL*.

[Zhou et al.(2022)] Kun Zhou, Yeyun Gong, Xiao Liu, Wayne Xin Zhao, Yelong Shen, Anlei Dong, Jingwen Lu, Rangan Majumder, Ji-Rong Wen, and Nan Duan. 2022. SimANS: Simple Ambiguous Negatives Sampling for Dense Text Retrieval. In *Proceedings of EMNLP Industry Track*.

A Per-seed results

Each cell gives recall at 1, 10, and 100 on the validation set for one run. The runs at learning rate 10^{-5} are the main results (Table 1). The runs at 3×10^{-5} are my first attempt, in which two runs collapsed.

Table 6: Learning rate 10^{-5}.

Sampler	Seed 0	Seed 1	Seed 2
In-batch	36.0 / 71.7 / 91.2	31.4 / 67.4 / 88.1	33.4 / 69.7 / 90.1
Uniform	64.4 / 92.3 / 98.5	64.5 / 92.4 / 98.4	64.1 / 92.0 / 98.4
Top-score	64.4 / 86.5 / 95.6	64.6 / 86.7 / 95.4	67.0 / 88.5 / 96.7
ReSa2	70.0 / 93.2 / 98.4	69.4 / 93.3 / 98.5	69.3 / 93.4 / 98.6

Table 7: Learning rate 3×10^{-5} (first attempt).

Sampler	Seed 0	Seed 1	Seed 2
In-batch	24.4 / 57.6 / 82.9	22.7 / 56.4 / 83.1	25.0 / 60.7 / 85.0
Uniform	60.7 / 90.0 / 98.0	61.5 / 90.3 / 97.9	61.1 / 90.4 / 97.8
Top-score	55.2 / 78.3 / 90.1	0.0 / 0.1 / 0.9	57.8 / 80.6 / 92.3
ReSa2	68.3 / 92.0 / 98.0	0.0 / 0.1 / 1.2	67.5 / 92.1 / 98.1

The recovery test continued the collapsed top-score seed-1 model for 1,500 more updates: with ReSa2 negatives it scored 0.0 / 0.0 / 2.2, with uniform negatives 0.0 / 0.0 / 1.1.

B The Lean file

Two files in the project repository, Lean 4 v4.34.1, built with `lake build`. `lean/LossBound.lean` uses Mathlib (real numbers, exp, log); its proofs depend only on the axioms `propext`, `Classical.choice` and `Quot.sound`. `lean/FalseNegatives.lean` uses the core library only.

LossBound.lean

```
import Mathlib.Analysis.SpecialFunctions.Log.Basic
import Mathlib.Algebra.BigOperators.Fin

/-!
# Easy negatives add almost nothing to the loss

Uses Mathlib for the real numbers, `Real.exp`, and `Real.log`. Check with:
`cd lean && lake build`.

Setup. One training mention has a correct page with score `sPos` and `n`
negatives with scores `s i`. The retriever loss (softmax cross entropy) is

  L = -log (e^{sPos} / (e^{sPos} + Σ e^{s i})).

The gap of negative `i` is `g i = sPos - s i`: how far it scores below the
correct page.

Results.
* `loss_eq_log_gaps`: L = log (1 + Σ e^{-g i}). The loss depends only on
  the gaps.
* `loss_le_sum_exp_neg_gap`: L ≤ Σ e^{-g i}, because log (1 + x) ≤ x.
  So a negative with gap g adds at most e^{-g} to the bound.
```

```
* ‘loss_nonneg‘: L ≥ 0.
-/

open Real Finset

/-- The softmax cross-entropy loss of one mention: correct-page score ‘sPos‘,
negative scores ‘s 0, ..., s (n-1)‘. -/
noncomputable def loss {n : ℕ} (sPos : ℝ) (s : Fin n → ℝ) : ℝ :=
  -Real.log (Real.exp sPos / (Real.exp sPos + ∑ i, Real.exp (s i)))

/-- The loss written with the gaps ‘sPos - s i‘. -/
theorem loss_eq_log_gaps {n : ℕ} (sPos : ℝ) (s : Fin n → ℝ) :
    loss sPos s = Real.log (1 + ∑ i, Real.exp (-(sPos - s i))) := by
  have hpos : 0 < Real.exp sPos := Real.exp_pos sPos
  have hsum : 0 ≤ ∑ i, Real.exp (s i) :=
    Finset.sum_nonneg fun i _ => (Real.exp_pos (s i)).le
  -- e^{-(sPos - s i)} = e^{s i} / e^{sPos}
  have hgap : ∀ i, Real.exp (-(sPos - s i)) = Real.exp (s i) / Real.exp sPos := by
    intro i
    rw [neg_sub, Real.exp_sub]
  -- 1 + Σ e^{-g i} = (e^{sPos} + Σ e^{s i}) / e^{sPos}
  have hrw : 1 + ∑ i, Real.exp (-(sPos - s i))
      = (Real.exp sPos + ∑ i, Real.exp (s i)) / Real.exp sPos := by
    rw [eq_div_iff hpos.ne', add_mul, one_mul, Finset.sum_mul]
    congr 1
    refine Finset.sum_congr rfl fun i _ => ?_
    rw [hgap i, div_mul_cancel₀ _ hpos.ne']
  rw [hrw, loss, ← Real.log_inv, inv_div]

/-- The loss is never negative. -/
theorem loss_nonneg {n : ℕ} (sPos : ℝ) (s : Fin n → ℝ) : 0 ≤ loss sPos s := by
  rw [loss_eq_log_gaps]
  apply Real.log_nonneg
  have : 0 ≤ ∑ i, Real.exp (-(sPos - s i)) :=
    Finset.sum_nonneg fun i _ => (Real.exp_pos _).le
  linarith

/-- Main bound: L ≤ Σ e^{-g i}. Each negative adds at most e^{-g} to it. -/
theorem loss_le_sum_exp_neg_gap {n : ℕ} (sPos : ℝ) (s : Fin n → ℝ) :
    loss sPos s ≤ ∑ i, Real.exp (-(sPos - s i)) := by
  rw [loss_eq_log_gaps]
  set x := ∑ i, Real.exp (-(sPos - s i)) with hx
  have hx0 : 0 ≤ x := Finset.sum_nonneg fun i _ => (Real.exp_pos _).le
  -- log y ≤ y - 1 for y > 0, with y = 1 + x
  have := Real.log_le_sub_one_of_pos (by linarith : (0 : ℝ) < 1 + x)
  linarith

/-- Corollary: if every negative scores at least ‘g‘ below the correct page,
then L ≤ n · e^{-g}. -/
theorem loss_le_of_gap_ge {n : ℕ} (sPos g : ℝ) (s : Fin n → ℝ)
    (hg : ∀ i, g ≤ sPos - s i) :
    loss sPos s ≤ n * Real.exp (-g) := by
  calc loss sPos s ≤ ∑ i, Real.exp (-(sPos - s i)) := loss_le_sum_exp_neg_gap sPos s
    _ ≤ ∑ _i : Fin n, Real.exp (-g) :=
        Finset.sum_le_sum fun i _ => Real.exp_le_exp.mpr (by linarith [hg i])
    _ = n * Real.exp (-g) := by simp
```

FalseNegatives.lean

The opening comment and the three examples at the end of the file are left out here.

```
/-- ‘s‘ scores above the correct page. -/
def above (sPos s : Int) : Bool := decide (sPos < s)

/-- Number of possible false negatives in a list of pages. -/
def nAbove (sPos : Int) (l : List Int) : Nat := l.countP (above sPos)
```

```
/-! ## 1. An upper bound for every sampler -/

/-- Any selection taken from the pool (each page used at most once) has at most
‘min (its size) A‘ possible false negatives. (Write the selected pages in
pool order; then "each page used at most once" means ‘sel‘ is a sublist.) -/
theorem any_sampler_le (sPos : Int) (pool sel : List Int) (h : sel.Sublist pool) :
    nAbove sPos sel ≤ min sel.length (nAbove sPos pool) :=
  Nat.le_min.mpr ⟨List.countP_le_length, h.countP_le⟩

/-! ## 2. The top-score rule reaches the bound

Sorting by score puts every page above ‘sPos‘ before every page below it.
So, for counting, top-score is: pages above first, then the others, take ‘k‘. -/

def topscore (sPos : Int) (k : Nat) (pool : List Int) : List Int :=
  (pool.filter (above sPos) ++ pool.filter (fun s => !above sPos s)).take k

/-- Taking ‘k‘ from (all-good list ++ no-good list) gives ‘min k (good length)‘ goods. -/
theorem countP_take_append (p : Int → Bool) (a b : List Int)
    (ha : ∀ x ∈ a, p x = true) (hb : ∀ x ∈ b, p x = false) :
    ∀ k, ((a ++ b).take k).countP p = min k a.length := by
  induction a with
  | nil =>
    intro k
    simp only [List.nil_append, List.length_nil, Nat.min_zero]
    exact List.countP_eq_zero.mpr fun x hx => by
      simp [hb x (List.mem_of_mem_take hx)]
  | cons x xs ih =>
    intro k
    cases k with
    | zero => simp
    | succ k =>
      have hx : p x = true := ha x (by simp)
      have ih' := ih (fun y hy => ha y (by simp [hy])) k
      simp only [List.cons_append, List.take_succ_cons, List.countP_cons, hx,
        ite_true, List.length_cons, ih']
      omega

theorem topscore_is_max (sPos : Int) (k : Nat) (pool : List Int) :
    nAbove sPos (topscore sPos k pool) = min k (nAbove sPos pool) := by
  unfold nAbove topscore
  rw [countP_take_append (above sPos) _ _ (fun x hx => (List.mem_filter.mp hx).2)
      (fun x hx => by simpa using (List.mem_filter.mp hx).2)]
  rw [List.countP_eq_length_filter]

/-! ## 3. Stage 2 of ReSa2 picks 7 of the top 75 uniformly

Counting fact: of all 7-page subsets of 75 pages, the share that contains one
fixed page is 7/75. So P(page selected) = (7/75) · P(page is in the top 75),
which is at most (7/75) · P(page is kept by stage 1). -/

/-- Binomial coefficient. Step i: C(n, i+1) = C(n, i) · (n − i) / (i + 1),
    and each division is exact. -/
def choose (n k : Nat) : Nat :=
  (List.range k).foldl (fun c i => c * (n - i) / (i + 1)) 1

/-- Subsets that contain the fixed page: choose the other 6 from 74.
    ‘75 · C(74,6) = 7 · C(75,7)‘, that is, the share is 7/75. -/
theorem stage2_share : 75 * choose 74 6 = 7 * choose 75 7 := by decide
```